Лекция 13. Интеграционные тесты репозитория

На прошлом уроке мы создали Data-слой нашего приложения (далее, для краткости, буду называть его "репозиторий"). Теперь нам необходимо его протестировать. что именно мы будем проверять?

При тестировании репозитория нам важно проверить, насколько правильно он работает с драйвером базы данных, схемой базы данных и SQL. Без взаимодействия с базой данных, тестировать репозиторий не имеет смысла.

Как вы поните из предыдущих уроков, тесты, в которых проверяется взаимодейтвие различных компонентов системы (напирмер: базы данных и нашего приложения), называются интеграционными.

repository.go × | repository_test.go | fixtures.go | database.go

```go
package employee

import (
  "github.com/jmoiron/sqlx"
)

type Repository struct {
  db *sqlx.DB
}

func NewRepository(database *sqlx.DB) *Repository {
  return &Repository{db: database}
}

func (r *Repository) FindById(id int64) (employee Entity, err error) {
  err = r.db.Get(&employee, "select * from employee where id = $1", id)
  return employee, err
}

func (r *Repository) Save(employee *Entity) (employeeId int64, err error) {
  err = r.db.Get(
    &employeeId,
    `insert into employee (name) values ($1) returning id`,
    employee.Name,
  )
  return employeeId, err
}

```

На скриншоте показан репозиторий с двумя методами, работ

которых мы будем тестировать.
Для начала, нам необходимо создать фикстуру. Фикстура - это код, который подготавливает окружение для запуска тестов. Почему полезно делать фикстуры. Если у вас много однотипных тестов (а это почти всегда так), то фикстура может быть переиспользована во всех тестах. Это избавит вас от необходимости дублировать код в тестах и позволит писать более чистые и понятные тесты.

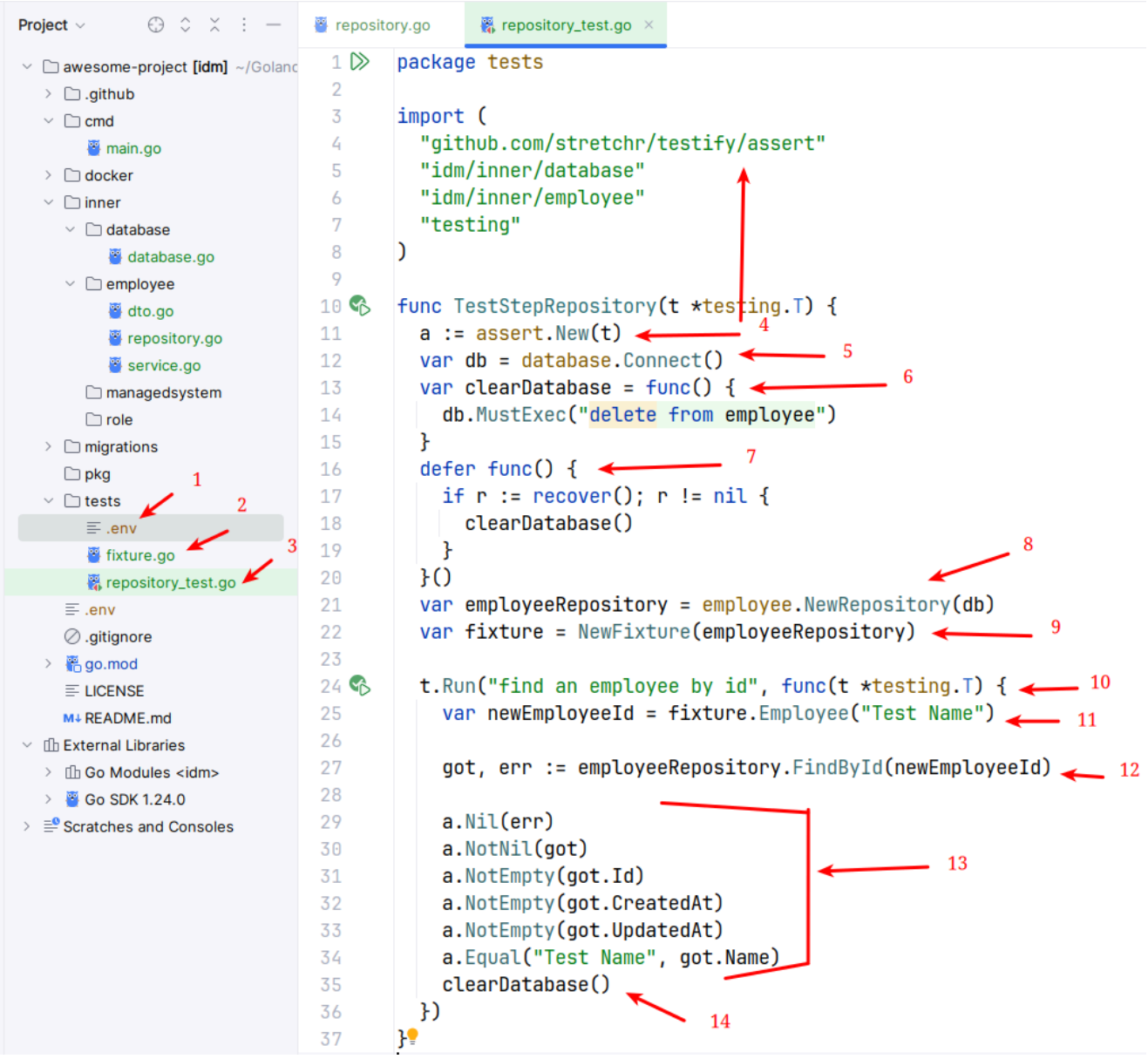
repository.go | repository_test.go | fixture.go | database.go

```go
package tests

import (
  "idm/inner/employee"
)

type Fixture struct {
  employees *employee.Repository
}

func NewFixture(employees *employee.Repository) *Fixture {
  return &Fixture{employees}
}

func (f *Fixture) Employee(name string) int64 {
  var entity = employee.Entity{
    Name: name,
  }
  var newId, err = f.employees.Save(&entity)
  if err != nil {
    panic(err)
  }
  return newId
}
```

"Назвался груздем - полезай в кузов". Мы начали делать нашеприложение в ООП стиле. Поэтому и фикстуру напишем тоже в ООП стиле. Но на вашем реальном проекте может быть принято заправило писать структуры в функциональном или процедурном стиле.
На скриншоте:
(1) - структура с кодом фикстуры
(2) - функция-конструктор, принимающая на вход employee.Repository (3) для работы с employee.Entity

(4) - метод для создания в бзе данных записи в таблице employee. Метод принимает на вход имя работника (5) (это единственное поле, которое может быть задано ползователем ашего приложния) и возвращает id записи, созданной им в таблице employee (6).

Вот так будет выглядеть наш тест:



(1) - .env - файл с конфигурацией подключения к тестовой базе данных. При запуске тестов, директорией запуска приложения является директория, где лежат тесты

(2) - фикстуры

(3) - интеграционный тест (лежит в отдельном пакете tests, чтобы можно было запускать его отдельно от юнит-тестов)

(4) - функции асертов

(5) - подключаемся к базе данных

(6) - функция очистки базы данных. Должна вызываться после каждог отеста

(7) - функция для очистки базы данных в случае паники
(8) - создаём репозиторий
(9) - создаём фикстуру
(10) - вложенный тест. В одной тестовой функции может быть много таких вложенных тестов. Это удобно, чтобы не повторять один и тот же код инициализации тестового окружения во многих тестах. Вложенный тест позволяет коротко описать, что он тестирует. Это удобно при чтении тестов
(11) - создаём запись в таблице employee в нашей тестовой базе данных
(12) - вызываем метод репозитория FindById, который тестируем в этом тесте
(13) - проверяем полученные данные
(14) - в конце теста вызываем функцию очистки базы данных.